

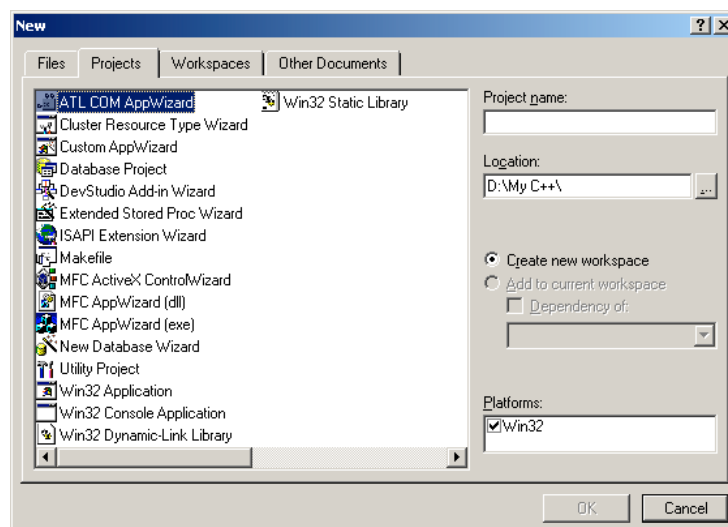
Digital Signature

(Programmer Manual)

ขั้นตอนการสร้างโปรแกรมแบบโอแอลอีเซิร์ฟเวอร์ (OLE Server)

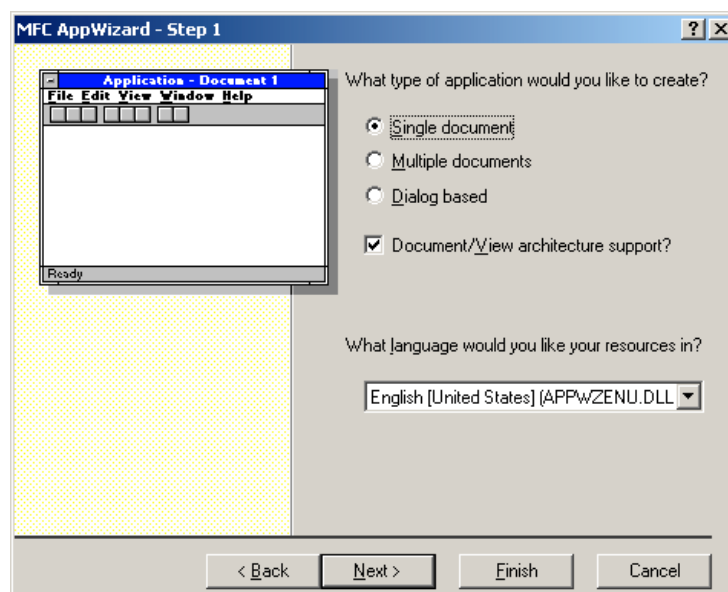
จากการออกแบบโปรแกรมคณะผู้จัดทำใช้โปรแกรมไมโครซอฟท์วิสวลซีพลัสพลัส (Microsoft Visual C++) ในการพัฒนา รูปแบบของโปรแกรมจะเป็นแบบโอแอลอีที่สามารถสร้างวัตถุหลายมือชื่อดิจิตอลแทรกลงในเอกสารไมโครซอฟท์เวิร์ดและไมโครซอฟท์เอกเซลได้ คณะผู้จัดทำจึงขออธิบายขั้นตอนการสร้างโปรแกรมต่อไปนี้

1. เมื่อทำการลงโปรแกรมไมโครซอฟท์วิสวลซีพลัสพลัสแล้วให้เลือก **File > New > Project** ซึ่งจะปรากฏไดอะล็อกดังต่อไปนี้



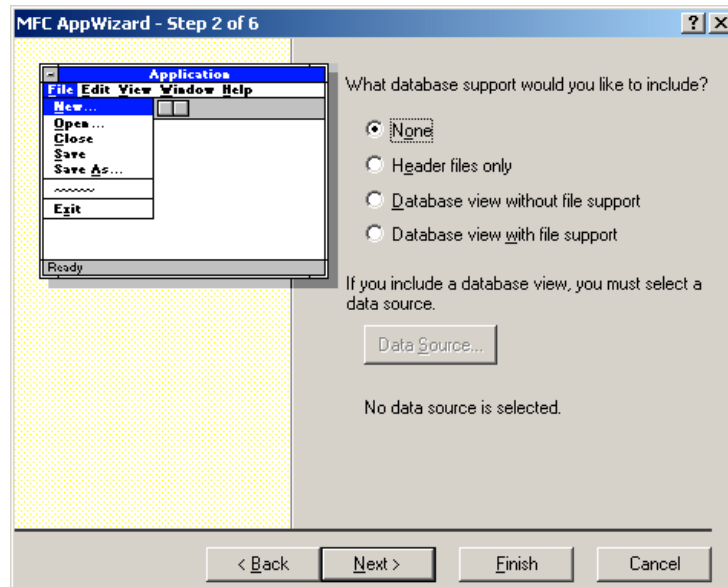
แล้วเลือก **MFC AppWizard** และ ที่ช่อง **Project Name** ให้พิมพ์ชื่อของโปรเจก ส่วนที่เหลือให้กำหนดตามที่โปรแกรมตั้งค่าไว้ จากนั้นกดปุ่ม **OK**

2. ในขั้นตอนที่ 2 จะปรากฏไดอะล็อก **MFC AppWizard - Step 1 of 6** แสดงดังต่อไปนี้



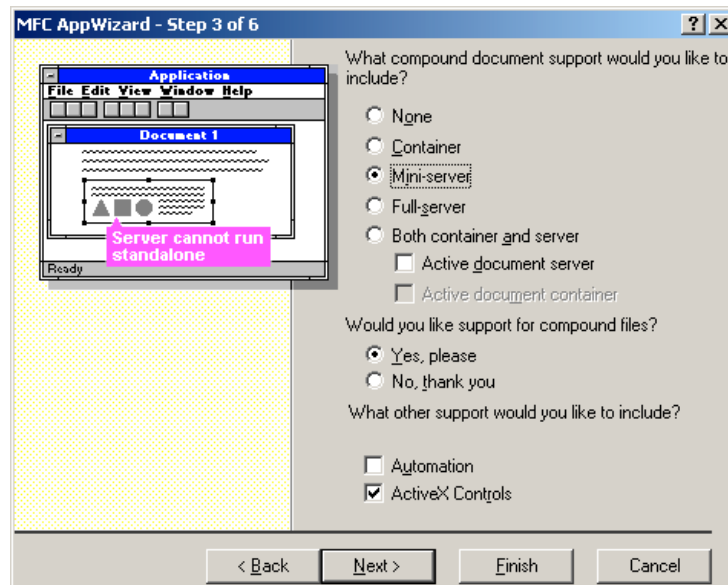
ให้เลือกลักษณะโปรแกรมแบบ **Single document** และสนับสนุนสถาปัตยกรรม **Document/View Architecture** จากนั้นกดปุ่ม **Next**

3. ในขั้นตอนที่ 3 จะปรากฏไดอะล็อก **MFC AppWizard - Step 2 of 6** แสดงดังต่อไปนี้



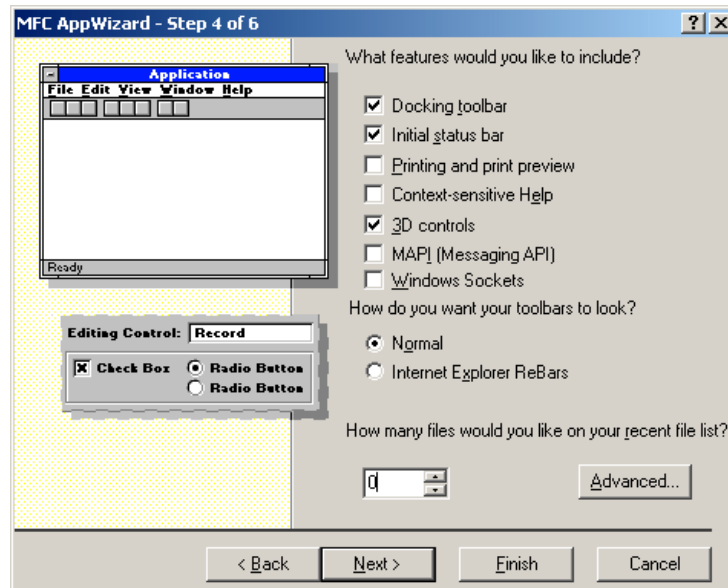
ในขั้นตอนนี้เป็นขั้นตอนให้เลือกการสนับสนุนการติดต่อกับดาต้าเบส ซึ่งโปรแกรมของเราไม่มีการติดต่อกับดาต้าเบสเลยให้เราเลือก **None** แล้วกดปุ่ม **Next** ได้เลย

4. ในขั้นตอนที่ 4 จะปรากฏไดอะล็อก **MFC AppWizard - Step 3 of 6** แสดงดังต่อไปนี้

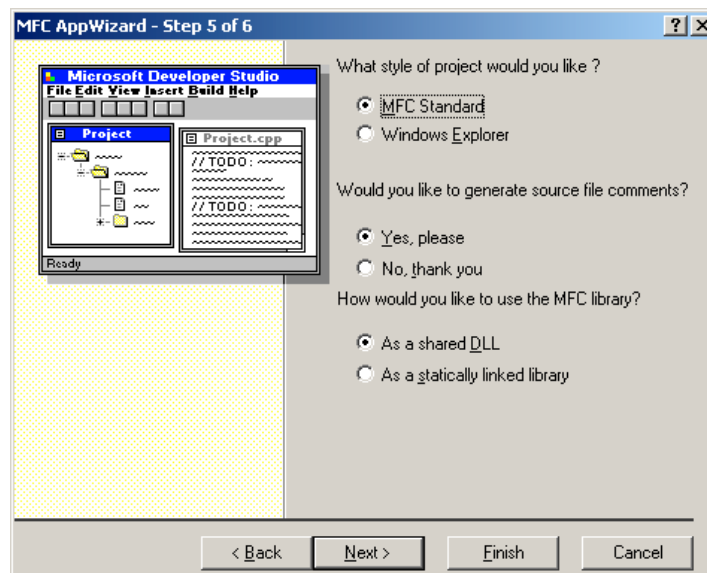


ขั้นตอนนี้เป็นขั้นตอนการเลือกให้โปรแกรมสนับสนุนการทำงานเป็นแบบโอแอลอี ให้เราเลือก **Mini - server** ที่เหลือให้ตั้งค่าตามที่โปรแกรมกำหนด แล้วกดปุ่ม **Next**

5. ในขั้นตอนที่ 5 จะปรากฏไคอะลอก MFC AppWizard - Step 4 of 6 แสดงดังต่อไปนี้ โดยให้เราตั้งค่าตามรูป จากนั้นกดปุ่ม Next



6. ในขั้นตอนที่ 5 จะปรากฏไคอะลอก MFC AppWizard - Step 5 of 6 แสดงดังต่อไปนี้ โดยให้เราตั้งค่าตามรูป จากนั้นกดปุ่ม Next

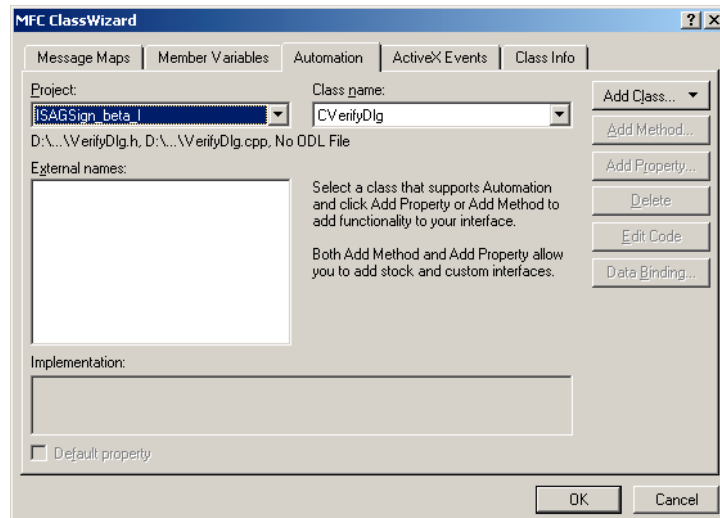


7. ในขั้นตอนที่ 5 จะปรากฏไคอะลอก MFC AppWizard - Step 6 of 6 ซึ่งเป็นขั้นตอนสุดท้าย ในขั้นตอนนี้ จะแสดงคลาสต่างๆ ที่โปรแกรมสร้างให้โดยอัตโนมัติ เมื่อเราทำการคลิกที่ คลาสต่างๆ สังเกตในช่อง Base class จะบอกว่าคลาสต่างๆนั้นสืบทอดมาจากคลาสใด พร้อมทั้งบอกชื่อไฟล์ source code (.cpp) และ ไฟล์ header (.h) อีกด้วย

ขั้นตอนการสร้างโปรแกรมแบบโอแอลอีออโตเมชัน (OLE Automation)

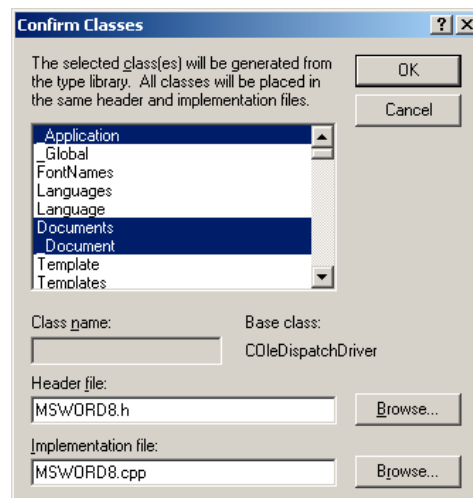
ขั้นตอนการสร้างโปรแกรมแบบโอแอลอีออโตเมชันนั้น เราจะใช้ในการเรียกเมธอดเพื่อดึงเอาข้อมูลในเอกสารไมโครซอฟท์เวิร์ดมาทำการสร้างลายมือชื่อดิจิทัล คณะผู้จัดทำขออธิบายรายละเอียดดังนี้

1. ที่เมนูบาร์เลือก **View > ClassWizard** หรือ กดปุ่ม **Ctrl – W** จะปรากฏไดอะล็อกดังรูปต่อไปนี้



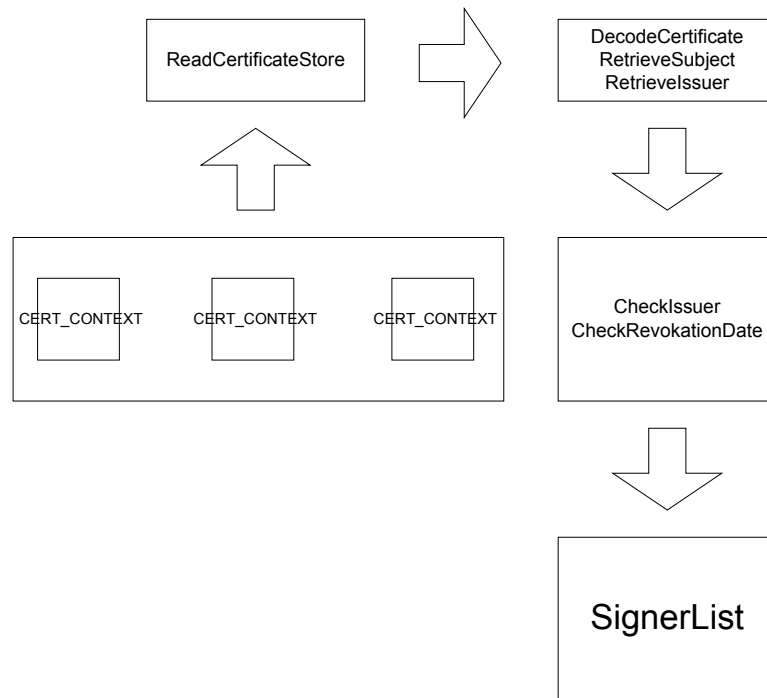
2. เลือกที่แท็บ Automation และคลิกที่ปุ่ม Add Class แล้วเลือก From a type library... จะปรากฏไดอะล็อกให้เราทำการเลือกไฟล์ขึ้นมา ในขั้นตอนนี้เราสามารถเลือกไฟล์ที่เป็น type library ของโปรแกรมต่างๆ ที่สนับสนุนโอแอลอีออโตเมชันได้ ซึ่งในที่นี้คือโปรแกรมไมโครซอฟท์เวิร์ด

ให้เราเลือกไฟล์ MSWORD8.OLB ซึ่งเป็นไฟล์ type library ของโปรแกรมไมโครซอฟท์เวิร์ด 97 แต่ถ้าเราต้องการให้โปรแกรมสามารถใช้โอแอลอีออโตเมชันของไมโครซอฟท์เอกเซลได้ ก็ให้เราเลือกที่ไฟล์ EXCEL8.OLB ได้เช่นเดียวกัน เมื่อเลือกไฟล์ดังกล่าวแล้วให้กดปุ่ม **OK** จะปรากฏไดอะล็อกดังต่อไปนี้



ในช่อง The selected class(es)... ให้เราเลือก **_Application** , **Documents** และ **_Document** ซึ่งเป็น
การระบุว่าเราต้องการจะใช้เมธอดจากคลาสใดของโปรแกรม จากนั้นกดปุ่ม **OK**

อธิบายฟังก์ชันต่างๆ ในส่วนที่ติดต่อกับที่เก็บเอกสารสิทธิ์ และการตั้งค่าพารามิเตอร์ต่างๆให้กับวัตถุ
มือชื่อดิจิตอล



เมื่อมีการเรียกใช้โปรแกรมเพื่อทำการสร้างลายมือชื่อดิจิตอล สิ่งที่เราพบครั้งแรกคือไดอะล็อกที่แสดงรายชื่อของเอกสารสิทธิ์ที่ได้รับรองจากองค์กรพิสูจน์สิทธิ์และเอกสารสิทธิ์นั้นต้องใช้งานได้ไม่หมดอายุ การทำงานของโปรแกรมในส่วนนี้จะเกี่ยวข้องกับคลาส `CCertStore` และโครงสร้างข้อมูล `SignerInformation` ซึ่งมีรายละเอียดดังต่อไปนี้

1. ฟังก์ชัน `ReadCertificateStore` จะการอ่านเอกสารสิทธิ์ทั้งหมดในที่เก็บเอกสารสิทธิ์แบบวน
ลูปไปเรื่อยๆ จนกว่าจะหมด

```
void CCertStore::ReadCertificateStore() {.....}
```

2. ถอดรหัสแต่ละเอกสารสิทธิ์ที่อ่านมาได้ในข้อ 1 ด้วยฟังก์ชัน `DecodeCertificate` แล้วทำการ
ตัดเอาเฉพาะฟิลด์ Subject (ระบุด้วย 2.5.4.3) และ Issuer (ระบุด้วย 2.5.4.10) ด้วยฟังก์ชัน
`RetrieveSubject` และ `RetrieveIssuer` ดังนี้

```
tempIssuerStr = DecodeCertificate(&(pCertContext->pCertInfo->Issuer));
```

```
tempSubjectStr = DecodeCertificate(&(pCertContext->pCertInfo->Subject));
```

```
tempIssuerStr = RetrieveIssuer(tempIssuerStr);
```

```
tempSubjectStr = RetrieveSubject(tempSubjectStr);
```

3. ทำการตรวจสอบว่าเอกสารสิทธิ์นั้นมีการรับรองจากองค์กรพิสูจน์สิทธิ์หรือไม่

```
if (tempIssuerStr != "NONE ISSUER") {
```

```
.....
```

```
.....
```

```
}
```

ทำการตรวจสอบว่าเอกสารสิทธิ์นั้นหมดอายุหรือไม่ด้วยฟังก์ชัน CheckRevocationDate

```
BOOL CCertStore ::CheckRevocationDate(const CERT_CONTEXT *pCertContext)
```

```
{ ..... }
```

4. ถ้าเอกสารสิทธิ์นั้นถูกต้อง คือ ได้รับการรับรองจากองค์กรพิสูจน์สิทธิ์และไม่หมดอายุ ก็ให้เก็บข้อมูลต่างลงในโครงสร้างข้อมูล SignerListInformation สิ่งที่มีดังนี้

SignerName : ชนิดข้อมูล CString

IssuerName : ชนิดข้อมูล CString

IssueDate : ชนิดข้อมูล CTime

RevocationDate : ชนิดข้อมูล CTime

ข้อมูลของเอกสารสิทธิ์ต่างๆ จะถูกนำมาแสดงในไดอะล็อก Sign Signature Step – 01 ซึ่งเป็นหน้าที่ของคลาส CSigStep01Dlg ซึ่งโค้ดส่วนดังกล่าวสามารถดูได้จากโค้ดของฟังก์ชัน OnInitDialog() ภายในคลาส CSigStep01Dlg

เมื่อผู้ใช้ทำการเลือกเอกสารสิทธิ์ใดๆ ที่แสดงแล้วกดปุ่ม Next เพื่อทำขั้นตอนต่อไป ข้อมูลที่อยู่ใน SignerInformation ของเอกสารสิทธิ์ที่เลือกจะถูกกำหนดให้กับคลาส SignatureInformation เพื่อประกอบเป็นข้อมูลลายมือชื่อ ซึ่งโค้ดสามารถดูได้จากฟังก์ชัน OnNext() ซึ่งรับ Event การกดปุ่ม Next ของ คลาส CSigStep01Dlg มีดังนี้

```
void CSigStep01Dlg::OnNext()
```

```
{
```

```
// TODO: Add your control notification handler code here
```

```
int selectNumber = m_signerList.GetSelectionMark();
```

```
if (selectNumber != -1)
```

```
{
```

```
// Get the name of selected signer.
```

```
signerSelectedName = m_signerList.GetItemText(selectNumber , 0);
```

```
theApp.currentSigner->setSignerInfo(signerList[selectNumber]);
```

```

        // Sign signature goto 2nd step.
        theApp.signStep = STEP02;
        // Close dialog.
        SendMessage(WM_CLOSE);
    }
    else
        // Show the error message.
        AfxMessageBox("ERROR..." , MB_OK | MB_ICONSTOP);
}

```

ขั้นตอนที่สองเป็นขั้นตอนการเลือกแฮชอัลกอริทึม และระยะเวลาของการรับรองลายมือชื่อดิจิตอลนี้ ข้อมูลต่างๆจะถูกกำหนดโดยผู้ใช้ เช่น กำหนดแฮชอัลกอริทึมเป็น SHA , ระยะเวลารับรองลายมือชื่อคือ 1 ม.ค. – 31 ธ.ค. 2546 เป็นต้น นอกจากนี้ยังเป็นการบันทึกว่าลายมือชื่อดิจิตอลนั้นสร้างขึ้น ณ เวลาใด โดยเวลาจะอยู่ในรูปแบบ UTC

ข้อมูลต่างๆ ในขั้นตอนนี้จะถูกเก็บในคลาส SignatureInformation ที่ต่อเนื่องมาจากขั้นตอนที่ 1 สามารถดูโค้ดโปรแกรมได้ที่ฟังก์ชัน OnSign() ของ คลาส CsignStep02Dlg

```

// Step 01 : Set hash algorithm.
if (!(m_Hash.IsEmpty()))
    theApp.currentSigner->setHashAlgorithm(m_Hash);
else
{
    AfxMessageBox("ERROR..." , MB_OK | MB_ICONSTOP);
    return;
}

// Step 02 : Set approve duration.
CTime certFm = m_fmdate;
CTime certTo = m_todate;
if (m_dur.GetCheck() == 1)
    theApp.currentSigner->setCertDuration(certFm, certTo);
else
    theApp.currentSigner->setCertDuration(NULL, NULL);

```



```
// Step 03 : Set sign signature date.

CTime signDt = CTime::GetCurrentTime();

theApp.currentSigner->setSignDate(signDt);

// Sign signature goto 3rd step.

theApp.signStep = STEP03;

// Close dialog.

SendMessage(WM_CLOSE);
```

อธิบายฟังก์ชันต่างๆ ที่ใช้ในขั้นตอนการสร้างลายมือชื่อดิจิตอล

1. การดึงข้อมูลจากเอกสารมาสร้างลายมือชื่อดิจิตอลโดยใช้ OLE Automation

เมื่อเราทำการสร้างคลาสที่ Import class จาก Type Library เรียบร้อยแล้วทั้งไมโครซอฟท์เวิร์ดและไมโครซอฟท์เอกเซล เราสามารถเรียกใช้ฟังก์ชันเพื่อดึงเอาข้อมูลที่อยู่ในเอกสารมาสร้างลายมือชื่อดิจิตอลได้ ขั้นตอนการเรียกใช้เมธอดต่างๆ นั้นมีดังนี้

Microsoft Word

1. ค้นหาหมายเลขคลาสและอินเตอร์เฟส IUnknown

```
// Find class id and IUnknown interface of MS-WORD Application.

CLSIDFromProgID(L"Word.Application" , &classID);

hResult = GetActiveObject((const CLSID&)classID , NULL , &pUnk);

if (hResult != S_OK)

    HandleError("Error during GetActiveObject !");
```

2. คิวรีอินเตอร์เฟส IDispatch

```
// Query IDispatch interface.

hResult = pUnk->QueryInterface(IID_IDispatch , (void*)&pDisp);

if (hResult != S_OK)

    HandleError("Error during QueryInterface for IDispatch !");
```

```
appObj.AttachDispatch(pDisp , TRUE);
```

```
IDisp = appObj.GetDocuments();
```

```
docObj.AttachDispatch(IDisp , TRUE);
```

3. คิวรีอินเตอร์เฟส IDataObject ซึ่งในขั้นตอนนี้จะได้วัตถุประเภท COleDataObject ที่มีวัตถุต่างๆ เช่น CF_TEXT , CF_BITMAP เป็นต้น

```
// Query IDataobject interface.
```

```
IDisp = appObj.GetActiveDocument();
```

```
docObj.AttachDispatch(IDisp,TRUE);
hResult = IDisp->QueryInterface(IID_IDataObject,(void**)&pDataOb);
if ( hResult != S_OK )
    HandleError("Error during QueryInterface for IDataObject !");
```

4. ตัดเอาเฉพาะข้อมูลที่เป็น CF_TEXT

```
// Get data object in CF_TEXT format from MS-WORD.
hResult = pDataOb->GetData(&etc,&stg);
```

Microsoft Excel

1. ค้นหาหมายเลขคลาสและอินเตอร์เฟส Iunknown

```
CLSIDFromProgID(L"Excel.Application" , &classID);
// Find class id and IUnknown interface of MS-EXCEL Application.
hResult = GetActiveObject((const CLSID&)classID , NULL , &pUnk);
if (hResult != S_OK)
    HandleError("Error during GetActiveObject !");
```

2. คิวรีอินเตอร์เฟส IDispatch

```
// Query IDispatch interface.
hResult = pUnk->QueryInterface(IID_IDispatch , (void**)&pDisp);
if (hResult != S_OK)
    HandleError("Error during QueryInterface for IDispatch !");
```

```
appObj.AttachDispatch(pDisp , TRUE);
IDisp = appObj.GetWorkbooks();
wrkObj.AttachDispatch(IDisp , TRUE);
```

3. คิวรีอินเตอร์เฟส IDataObject ซึ่งในขั้นตอนนี้จะได้ว่าวัตถุประเภท COleDataObject ที่มีวัตถุ ต่างๆ เช่น CF_TEXT , CF_BITMAP เป็นต้น

```
// Query IDataobject interface.
IDisp = appObj.GetActiveWorkbook();
wrkObj.AttachDispatch(IDisp , TRUE);
hResult = IDisp->QueryInterface(IID_IDataObject,(void**)&pDataOb);
if ( hResult != S_OK )
    HandleError("Error during QueryInterface for IDataObject !");
```

4. ตัดเอาเฉพาะข้อมูลที่เป็น CF_TEXT

```
// Get data object in CF_TEXT format from MS-EXCEL.
hResult = pDataOb->GetData(&etc,&stg);
```

ขั้นตอนต่อไป คือ การแปลงข้อมูลประเภท CF_TEXT ให้เป็นบิตสตรีม ซึ่งมีขั้นตอนดังนี้

```
HGLOBAL hgData = stg.hGlobal;

BYTE *dataBlock = (BYTE*)GlobalLock(hgData);

และหาความยาวของบิตสตรีมนั้น ดังนี้

UINT dataBlockLen = GlobalSize(stg.hGlobal);
```

2. การสร้างลายมือชื่อดิจิตอลและตรวจสอบลายมือชื่อดิจิตอล

การทำงานในส่วนนี้เป็นหน้าที่ของคลาส CSigAndVerify ซึ่งมีฟังก์ชันต่างๆ ที่สำคัญดังนี้

FUNCTION_RESULT CVerifySignature::FindHashValue(BYTE* dataStream , UINT dataStreamLen , CString hashAlgo)

```
{
    // เป็นฟังก์ชันที่ทำหน้าที่ในการหาแฮชจากข้อมูลที่เป็นบิตสตรีมที่ส่งเข้ามา ผลลัพธ์ที่
    // ได้ คือ Hash Object ซึ่งจะถูเก็บแบบชั่วคราวใน CSP
    // Input      : dataStream เป็นสตรีมบิตข้อมูลที่ต้องการนำมาหาแฮชเชิงไคเจส
    //            : dataStreamLen เป็นความยาวของสตรีมบิตข้อมูล
    //            : hashAlgo เป็นแฮชอัลกอริทึมที่ใช้ในการหาแฮชเชิงไคเจส
    // Output     : ถ้าฟังก์ชันทำงานสำเร็จจะ return SUCCESSFUL แต่ถ้าเกิดข้อผิดพลาด
    //            : ระหว่างที่ฟังก์ชันทำงานอยู่จะ return ค่าข้อผิดพลาดต่างๆที่พบ
    .....
}
```

FUNCTION_RESULT CVerifySignature :: GetHashValue()

```
{
    // เป็นฟังก์ชันที่ทำหน้าที่แปลงค่าแฮชเชิงไคเจสที่อยู่ใน Hash Object ให้เป็นข้อมูลประเภท
    // CString
    // Input      : -
    // Output     : ถ้าฟังก์ชันทำงานสำเร็จจะ return SUCCESSFUL แต่ถ้าเกิดข้อผิดพลาด
    //            : ระหว่างที่ฟังก์ชันทำงานอยู่จะ return ค่าข้อผิดพลาดต่างๆที่พบ
    .....
}
```

FUNCTION_RESULT CVerifySignature::VerifySignature(BYTE *message , DWORD messageLen)

```
{
    // Input      : message เป็นลายมือชื่อดิจิตอลที่รับรองเอกสารในรูปแบบของบิตสตรีม
```

```

//      : messageLen เป็นความยาวของลายมือชื่อดิจิตอล
// Output : ถ้าค่าของเมสเสจไคเจสของลายมือชื่อกับเมสเสจไคเจสที่สร้างจาก
//      เอกสารตรงกันจะ return APPROVE แต่ถ้าไม่ตรงกันจะ return
//      UNAPPROVE และถ้าพบข้อผิดพลาด ระหว่างที่ฟังก์ชันทำงานอยู่จะ
//      return ค่าข้อผิดพลาดต่างๆที่พบ
// .....
}

```

FUNCTION_RESULT CSignSignature::SignSignature(CString signerName)

```

{
// ทำการสร้างลายมือชื่อดิจิตอลตามชื่อที่ระบุมา ฟังก์ชันนี้จะทำการติดต่อกับที่เก็บเอกสารสิทธิ์
// และคีย์คอนเทนเนอร์เพื่อที่จะนำคีย์ส่วนตัวมาเข้ารหัสสร้างลายมือชื่อดิจิตอล
// Input      : signerName เป็นชื่อของผู้ลงลายมือชื่อดิจิตอล ชื่อนี้จะถูกนำมาระบุตัว
//      เอกสารสิทธิ์ที่เก็บในที่เก็บเอกสารสิทธิ์และคีย์ส่วนตัวที่สอดคล้องกัน
// Output     : ถ้าฟังก์ชันทำงานสำเร็จจะ return SUCCESSFUL แต่ถ้าเกิดข้อผิดพลาด
//      : ระหว่างที่ฟังก์ชันทำงานอยู่จะ return ค่าข้อผิดพลาดต่างๆที่พบ
// .....
}

```

CString CSignSignature::GetCertificateSubject(CERT_NAME_BLOB *nameBlob)

```

{
// เป็นฟังก์ชันที่ทำหน้าที่ในการแปลงข้อมูลในฟิลด์ Subject ซึ่งฟิลด์ประเภทนี้จะเป็นข้อมูล
// ประเภท CERT_NAME_BLOB ของเอกสารสิทธิ์ (CERT_INFO) โดยการแปลงจะแปลงให้
// อยู่ในรูปแบบ CString
// Input      : nameBlob ข้อมูลในฟิลด์ Subject ของ เอกสารสิทธิ์ (CERT_INFO)
// Output     : ถ้าฟังก์ชันทำงานสำเร็จจะ return CString ที่แปลงออกมาได้ แต่ถ้าเกิดข้อผิดพลาด
//      ระหว่างที่ฟังก์ชันทำงานอยู่จะ return ค่าข้อผิดพลาดต่างๆที่พบในรูปแบบ
//      แบบที่เป็น CString
// .....
}

```

การออกแบบโปรแกรมในส่วนที่ติดต่อกับโอแอลอี

การออกแบบโปรแกรมในส่วนที่ติดต่อกับเอกสารกับวัตถุลายมือชื่อดิจิตอล มีดังนี้

1. CISAGSignDoc

คลาสนี้เป็นคลาสที่สืบทอดมาจากคลาส COleDocument ซึ่งสืบทอดมาจากคลาส CDocument อีกต่อหนึ่ง เป็นคลาสที่เก็บข้อมูลของลายมือชื่อดิจิตอลที่สร้างขึ้นในเอกสาร ได้แก่ ตัวแปรที่เป็นชนิดของคลาส CSignatureInformation การทำงานในคลาสนี้จะมีฟังก์ชันต่าง ๆ ที่สำคัญดังนี้

- **Serialize()** ฟังก์ชันทำหน้าที่ในการนำข้อมูลจากด็อกคิวเมนต์วัตถุลายมือชื่อและบันทึกข้อมูลของวัตถุลายมือชื่อดิจิตอลลงด็อกคิวเมนต์
- **OnNewDocument()** ฟังก์ชันในการสร้างด็อกคิวเมนต์หรือที่เก็บข้อมูลลายมือชื่อขึ้นมาใหม่

2. CISAGSignView

คลาสนี้เป็นคลาสที่สืบทอดมาจากคลาส CView เป็นคลาสที่ทำหน้าที่ในการแสดงรูปร่างลักษณะตัวโปรแกรมที่สร้างลายมือชื่อดิจิตอล คือเมื่อผู้ใช้ทำการดับเบิลคลิกที่ตัววัตถุลายมือชื่อดิจิตอล ตัวโปรแกรมโอแอลอีเซิร์ฟเวอร์จะถูกเรียกขึ้นมาส่วนที่เป็นเมนูบาร์ของโปรแกรมโอแอลอีคอนเทนเนอร์จะเปลี่ยนไปเป็นแบบโปรแกรมโอแอลอีเซิร์ฟเวอร์ซึ่งในส่วนนี้จะเป็นหน้าที่ของคลาส IpFrame ซึ่งจะกล่าวถึงต่อไป ส่วนที่เป็นพื้นที่แสดงผลจะเป็นส่วนของโปรแกรมโอแอลอีเซิร์ฟเวอร์เช่นกัน คลาสนี้จะดูแลการแสดงผลในส่วนนี้นั่นเอง

3. CISAGSignSrvrItem

เป็นคลาสที่ทำหน้าที่ในการดูแลการแสดงผลรูปวัตถุลายมือชื่อดิจิตอล ที่แทรกอยู่ในเอกสาร

- **OnDraw()** เป็นฟังก์ชันที่ทำหน้าที่แสดงผลของวัตถุลายมือชื่อ ว่าเอกสารนั้นมีการเปลี่ยนแปลงหรือไม่ โดยจะแสดงออกเป็น 3 รูปคือ
 - Digital Signature หมายถึง เอกสารนี้ยังไม่ได้ถูกตรวจสอบ
 - Unmodified Document หมายถึง เอกสารนี้ไม่มีการเปลี่ยนแปลง
 - Modified Document หมายถึง เอกสารนี้มีการเปลี่ยนแปลง